

INHN0006 — Mocking / Testing / REST practice project

A single Gradle project holding one runnable, self-contained example per exam scenario, plus a mutation drill that proves the tests can actually go red.

Built around one fact from your graded exams: **Artemis runs your tests against a deliberately broken implementation. A test that cannot fail is worth zero points.**

Quick start (Windows / PowerShell)

```
.\gradlew.bat build
```

```
.\gradlew.bat test
```

```
.\gradlew.bat mutationReport
```

Run one scenario:

```
.\gradlew.bat test --tests "ise.mocking.exam_vehicle.*"
```

Start a server:

```
.\gradlew.bat runProductServer
```

Task	What it does
test	The whole green suite (423 tests, 33 deliberately skipped)
mutationReport	Runs all 56 broken implementations and prints the catch table
runProductServer	Product REST server on http://localhost:8080
runPulloverServer	Retake-exam Pullover server on http://localhost:8081
runPracticeServer	REST practice drill server on http://localhost:8082
runTicketServer	ISE HN 2026 Ticket Manager server on http://localhost:8083

Import into IntelliJ IDEA as a **Gradle** project (File > Open → pick build.gradle → *Open as Project*).

Layout

```
src/main/java/ise/
  mocking/    s01_setup_annotations ... s17_errorcatalogue, exam_vehicle, exam_docking
  testing/    s01_lifecycle ... s17_boundaries, exam_library, exam_cart
  rest/       model/ server/ client/ exam_pullover/ practice/
src/test/java/ise/
  mocking/    the EasyMock tests
  testing/    the JUnit tests
  rest/       MockMvc endpoint tests + the WebFlux client test
  mutants/    56 broken implementations + their tagged test classes
  drill/      MutationDrillTest – runs every mutant, asserts each goes red
  practice/   gutted drills + generated solutions
docs/
  README.md      this file
  SCENARIO_INDEX.md every scenario, its package, and the one rule to remember
  EXAM_DRILLS.md  the mutation table – scenario → mutation → catching assertion
  MOCK_EXAM.md    the Jun 2026 mock exam: new format, worked answers, quiz key
  ISEHN_2026_EXAM.md the real Aug 2026 endterm: all 5 exercises solved + what the grader docked
  CHEAT_SHEET.md  one-file cheat sheet: JUnit, EasyMock, REST, black-box, UML, quiz, git, setup
  PRACTICE_QUESTIONS.md new questions in the 2026 shapes, with answers
  pdf/           every .md in docs/ as a searchable PDF (with an index)
  ISE_Master_Cheatsheet.md the long Ctrl+F master cheat sheet (lectures + all past exams, tags like
#easymock)
  pdfs/          the five original PDFs (Final Guide, black-box, if-in-test, component diagram,
BIE setup)
  templates/     10 copy-paste templates (JUnit, parameterized, EasyMock, REST, black-box,
Gherkin) + index
```

Plus three areas added by the June 2026 mock exam, which dropped REST entirely:

```
src/main/java/ise/
  solid/       exam_enrollment/    SOLID principles – 25 pts
  blackbox/    exam_gamelauncher/  equivalence classes + boundary values – 10 pts
  acceptance/  exam_login/          Gherkin acceptance tests – 5 pts
  mocking/     exam_smarthome/      the new mocking exercise – 25 pts
```

And the real August 2026 endterm (see `docs/ISEHN\_2026\_EXAM.md`):

src/main/java/ise/ testing/ exam_cloudserver/ implement 2 methods + 5 tests — 20 pts rest/
exam_ticket/ Ticket Manager, 5 endpoints — 25 pts (run: runTicketServer) blackbox/ exam_graphics/
equivalence classes + boundaries — 15 pts blackbox/ practice_streaming/ extra black-box practice

Environment

Thing	Version	Note
Java	compiled with --release 17	Your machine has JDK 21 installed; the release flag pins both bytecode and the visible API to 17, so nothing here can accidentally use a Java 18+ method that the exam machine lacks.
Gradle	8.14.5 (wrapper)	Generated from a distribution already cached on your machine.
JUnit Jupiter	5.10.2	via the Spring Boot BOM
EasyMock	5.2.0	
Spring Boot	3.2.5	starter-web, starter-webflux, starter-test

Thing	Version	Note
JUnit Platform Launcher	1.10.2	needed by the mutation drill only

Mockito is on the classpath as a transitive dependency of `spring-boot-starter-test`. Nothing in this project uses it — the exam is EasyMock.

Two build-file decisions worth knowing

No Spring Boot Gradle plugin. The plugin wants exactly one main class, and this project ships three runnable servers; a `bootJar` failure would block `gradlew test`, which is the one command that must never break. The Spring Boot **BOM** pins the identical library versions, so the code you write here compiles unchanged on Artemis. The two things the plugin would otherwise have given you are added by hand:

`options.compilerArgs << '-parameters'`. Without it, `@PathVariable Long productId` fails **at request time**, not at compile time:

Name for argument of type [java.lang.Long] not specified, and parameter name information not available via reflection. Ensure that the compiler uses the '-parameters' flag.

The other fix is naming every binding explicitly: `@PathVariable("productId")`. Worth knowing, because if the exam project ever shows you this error you will otherwise spend ten minutes staring at a correct-looking controller.

How the mutation drill works (Part 4)

You asked which of the two approaches I would pick. **I used both**, because the two areas have genuinely different shapes:

Testing scenarios → parameterised over the implementation. Each test class is `public` and builds its SUT through a `protected` factory method (`newRegistry()`, `newCart()`, ...). A mutant is a subclass in `ise.mutants.testing` that overrides *only* that factory:

```
@Tag("mutant")
class ClubRegistryStateTest_RegistersNobody extends ClubRegistryStateTest {
    @Override protected ClubRegistry newRegistry() {
        return new ClubRegistryThatRegistersNobody();
    }
}
```

The assertions are **inherited verbatim** — there is no way for me to have accidentally weakened them, which is the whole point.

Mocking scenarios → standalone mutant tests. The EasyMock record phase lives inside the test body, so there is nothing to inherit. Each mutant repeats the original expectations and points them at a broken SUT. And the exam mocking tests are deliberately written *literally* exam-shaped —

`@TestSubject private VehicleManagementSystem vehicleManagementSystem = new VehicleManagementSystem();` — because field injection cannot be parameterised and your muscle memory needs the exact text.

The runner. `ise.drill.MutationDrillTest` uses the JUnit Platform Launcher to execute each mutant class programmatically and asserts its failure count is `> 0`. Mutants carry `@Tag("mutant")` and

`build.gradle` excludes that tag from `test`, so they never pollute the green suite. Net effect:

- `gradlew.bat test` → green, **and the proof that your tests go red is part of that green**
- `gradlew.bat mutationReport` → same run, with the table printed

A mutant that *passes* fails the drill. That is the alarm you want.

Practice mode (Part 5)

`src/test/java/ise/practice/` holds the same exercises with the bodies gutted down to `// TODO:` comments carrying the **exam wording**, plus the graded feedback each task punished last time. Every practice class is `@Disabled` so the suite stays green.

To attempt one: delete the `@Disabled`, fill in the TODOs, run `gradlew.bat test --tests "ise.practice.testing.*"`, then diff against the matching class in `.../solutions/`.

The REST drill works the other way round, exactly like Artemis: the tests are given (`ise.practice.rest.PracticeStockResourceTest`) and you write the server in `ise.rest.practice`. I verified it is solvable — solved it, ran all 7 tests green, then restored the TODOs.

The solution classes are **generated** from the canonical exam test classes, so they can never drift away from the versions the mutation drill actually verifies.

What the graded feedback maps onto

Exam feedback	Where it is fixed
"your test for registering members passed, but <code>registerMember()</code> was broken"	<code>ise.testing.s12_voidstate</code> , <code>ise.testing.exam_library</code>
"your test for a new library ... did not correctly check for the number of members"	<code>LibraryTest.testInitialLibraryIsEmptyAndHasOneMember</code>
" <code>testAssignWithEngineFailure()</code> ... should check that <code>assign</code> is not called"	<code>ise.mocking.s14_mustnotbecalled</code> , <code>exam_vehicle</code>
" <code>testAssignDriverFailure()</code> does not fail on a wrong implementation"	<code>exam_vehicle</code> — the <code>verify(vehicleMock)</code> line
"make sure <code>testDockAstronautFailure()</code> fails if <code>dock()</code> is never executed"	<code>exam_docking</code> — same fix
"you did not configure the test subject correctly"	<code>ise.mocking.s01_setup_annotations</code> , both exam classes
<code>testInitialCartIsEmpty</code> → expected: <code><[]></code> but was: <code><0></code>	<code>exam_cart</code> — compare an int to an int
<code>testAddNullProductThrowsException</code> → the exception escaped	<code>exam_cart</code> — the call belongs inside the lambda
REST 0/22, no commit made	<code>ise.rest.*</code> — a complete, running server and client

Errors found in the playbook PDF

See the bottom of `SCENARIO_INDEX.md`. The short version: **`.times(0)` does not work**, and the playbook recommends it twice — including in section 14, the exam's hardest case.

Files in this folder that I did not touch

`src/Main.java`, `EveryTHING.iml` and `.idea/` were here before and are untouched. One heads-up: `EveryTHING.iml` declares `src` itself as a source root, which conflicts with the Gradle layout (`src/main/java`, `src/test/java`). IntelliJ will sort this out when you re-open the project as a Gradle project, but if you see duplicate-class warnings, that stale module file is why. The only pre-existing file I changed is `.gitignore`, where I **appended** a Gradle block (`build/`, `.gradle/`); nothing already in it was reformatted.